École Polytechnique Fédérale de Lausanne

Teleoperation for Anthropomorphic Robotic Manipulation using RL-Based Robust Retargeting

Master thesis

Author:

Jeffrey Yu, 371327

CREATE Lab, Prof. Josie Hughes
Supervisor: Cheng Pan

September 4, 2026

Contents

1 Introduction

Dexterous bimanual manipulation — tasks requiring two hands to coordinate precisely around a shared object — remains one of the most challenging problems in robotics. Contact-rich interactions demand not only accurate finger positioning but also the ability to adapt forces and contacts in real time. Classical motion planning approaches struggle to scale to the high dimensionality of two dexterous hands, while hand-coded controllers are brittle to variation in object placement or operator intent.

Reinforcement learning (RL) offers a principled alternative: policies trained in simulation can discover contact strategies that are difficult to specify manually. However, learning from scratch in such high-dimensional spaces is sample-inefficient. Human demonstrations address this by providing dense reference signals that reduce the exploration burden, turning the problem into trajectory tracking [1, 2, 3]. Recent work has demonstrated that RL policies guided by human hand-object demonstrations can achieve high-fidelity bimanual manipulation in simulation [1].

For real-world deployment, however, the policy must respond to *live* human input rather than a fixed recorded trajectory. At teleoperation time the incoming hand motion will inevitably differ from any trajectory seen during training — the operator may approach the object from a different angle, use a different grasp, or vary the timing of the motion. The central challenge is therefore *demonstration generalization*: can a single policy handle unseen trajectories of the same task, rather than being fitted tightly to one specific reference?

Existing approaches address this only partially. ManipTrans [1] and DexMachina [2] each train a separate policy per demonstration, fitting the residual controller to a single reference trajectory, and QuasiSim [4] optimizes each trajectory independently at a cost of tens of GPU-hours per sequence. A policy trained this way memorizes the specific kinematic path of that demonstration and is brittle when the live input deviates from it, which is unavoidable in teleoperation. DexTrack [3] and Dexplore [5] do train a single controller across many references, but both are driven by recorded motion-capture corpora rather than by live operator input, and neither targets the bimanual residual setting. This motivates a shift in training objective: instead of fitting to one demonstration, we train a single shared policy across a *set* of demonstrations that vary in starting configuration, approach direction, and grasp style, so that the policy learns the underlying manipulation skill rather than a specific trajectory.

To collect such demonstrations and evaluate live teleoperation in simulation, we build a hardware pipeline combining the Apple Vision Pro (AVP) — which provides 21 hand keypoints and wrist pose per hand at 60 Hz — with an OptiTrack motion capture system providing object pose. Because these two systems operate in independent coordinate frames, we perform a rigid-body spatial calibration using the Umeyama algorithm [6] to align them into a common reference frame before feeding observations into the simulator.

Contributions. The contributions of this paper are two-fold:

- **A bimanual teleoperation and data collection pipeline** integrating the Apple Vision Pro and OptiTrack motion capture, with cross-system spatial calibration via the Umeyama algorithm, for recording synchronized hand and object trajectories at 60 Hz and streaming them into the Isaac Gym simulator.
- **A single residual policy that generalizes across unseen demonstration trajectories**, demonstrated on a bimanual bottle-capping task. In contrast to prior work [1], which requires a separate policy per demonstration, our policy is trained jointly on multi-

ple demonstrations varying in starting configuration and grasp approach, and successfully teleoperates the task on trajectories not seen during training.

2 Related Work

2.1 Dexterous Manipulation via Reinforcement Learning

Reinforcement learning has produced strong results in high-dimensional dexterous manipulation. Large-scale RL with domain randomization enables cube reorientation with a multi-fingered hand [7], and DeXtreme extends this to real hardware through careful sim-to-real transfer [8]. These methods learn from scratch using task-specific reward functions, which requires extensive engineering and limits generalization to new tasks.

Reference-guided RL reduces this burden by providing a human demonstration as a tracking objective, turning long-horizon exploration into a trajectory-following problem [9]. Applied to dexterous hands, DexTrack trains a neural tracking controller from human references [3], and Dexplore uses reference-scoped exploration to cover diverse manipulation skills [5]. Dexplore [5] argues that this staged decomposition — retargeting, tracking and residual correction — leaves demonstrations underused and compounds error across stages, and replaces it with a single-loop optimization that treats the demonstration as soft guidance. We retain the staged design for a different reason: freezing the imitator yields a policy that runs against an arbitrary incoming target stream at control rate, which is what makes live teleoperation possible, whereas an optimization coupled to a demonstration corpus has no equivalent inference-time counterpart. ManipTrans [1] introduces a two-stage framework specifically for bimanual dexterous manipulation: a generalist hand imitator is pre-trained on large-scale hand motion data, and a residual policy is fine-tuned per demonstration to satisfy task-specific physical constraints. DexMachina [2] similarly trains per-demonstration policies with functional retargeting objectives. Both achieve strong performance on their training demonstrations but train a separate policy per reference trajectory, and therefore do not generalize to unseen trajectories of the same task. Our work builds on the ManipTrans framework and extends it to a multi-demonstration training setting, where a single shared residual policy is exposed to trajectories varying in starting configuration and grasp approach, enabling it to handle novel inputs at teleoperation time.

2.2 Teleoperation for Dexterous Manipulation

Teleoperation systems map live human hand observations to robot hand commands, enabling data collection and direct robot control. DexPilot formulates vision-based dexterous teleoperation around task-space fingertip objectives [10], and AnyTeleop generalizes this to multiple robot embodiments [11]. OpenTeach [12] and Bunny-VisionPro [13] demonstrate low-latency teleoperation using consumer hand-tracking hardware and ZMQ-based streaming pipelines. OpenTeleVision [14] extends this to immersive first-person teleoperation using a head-mounted display. BEAVR [15] and AnyTeleop [11] further study the role of retargeting quality in downstream task performance. TeleDexter [16], evaluated on single-hand rather than bimanual manipulation, targets human-level dexterous teleoperation and introduces a random action-masking augmentation, periodically freezing a subset of joint commands during training so that the policy cannot assume every degree of freedom responds on schedule; we adopt this augmentation, described in Appendix A.2.

A common pattern across these systems is direct joint retargeting: human joint angles are mapped to robot joint targets via kinematic correspondences and streamed to the robot at

control frequency, without a learned policy that adapts to contact forces. In contrast, our approach inserts a trained RL residual policy between the operator commands and the robot actions, providing compliance and contact correction that a retargeting-only pipeline cannot supply. Furthermore, while prior teleoperation systems are typically calibrated to a specific operator, our policy is trained from the outset on a distribution of demonstration styles — varying in starting configuration, approach direction, and grasp — rather than on a single reference trajectory.

2.2.1 Hand Retargeting

Central to teleoperation quality is the retargeting step that maps human hand observations to robot joint targets. Early approaches define hand-centric objectives over fingertip correspondences or joint alignment [10, 11], which align the robot hand with the human hand but do not directly specify where and how the robot should contact the object. Recent work addresses this gap by incorporating contact geometry into the retargeting objective. TopoRetarget [17] constructs a sparse interaction graph over hand and object keypoints and optimizes distance-weighted Laplacian deformation with directional consistency, achieving state-of-the-art contact preservation on the ContactPose dataset and significantly improving downstream RL tracking policy success rates on contact-rich tasks such as pen spinning. In our pipeline we do not apply a separate retargeting step at inference time; instead, the pre-trained imitator π_I absorbs the morphological gap, and the residual policy π_R handles the remaining contact adaptation.

2.3 Data Collection

Several systems have been developed for portable, high-quality hand-object data collection. DexCap [18] uses a wrist-mounted SLAM camera combined with OptiTrack markers to record hand-object interaction without fixed infrastructure. OakInk-V2 [19] captures rich bimanual manipulation sequences using optical motion capture and provides both MANO hand parameters and object poses. The ARCTIC dataset [20] focuses on articulated object manipulation with high-precision bimanual trajectories.

Our pipeline combines the Apple Vision Pro — a consumer XR headset that provides 21 MANO-compatible hand keypoints and wrist pose at 60 Hz via the `avp_stream` SDK — with OptiTrack Motive for rigid-body object pose tracking. In contrast to prior data collection pipelines [18, 19, 1, 2], which recover hand parameters through offline optimization-based retargeting after capture, the AVP provides MANO-compatible keypoint positions directly at capture time. This eliminates the post-processing stage entirely and allows the same hardware pipeline to be used for live teleoperation without modification.

3 Methods

3.1 RL Formulation

We consider a bimanual dexterous manipulation task in which a pair of robotic hands $\mathbf{d} = \{d_l, d_r\}$ must reproduce a manipulation demonstrated by human hands $\mathbf{h} = \{h_l, h_r\}$ interacting with two objects $\mathbf{o} = \{o_l, o_r\}$. In our target task, the left hand stabilizes an alcohol bottle while the right hand picks up and replaces the cap.

A human demonstration provides two reference signals: a hand trajectory $\mathcal{T}^h = \{\tau_h^t\}_{t=1}^T$ en-

coding the wrist 6-DoF pose, linear and angular velocity, and finger joint positions; and an object trajectory $\mathcal{T}^o = \{\tau_o^t\}_{t=1}^T$ encoding the 6-DoF object's pose and velocity. All translations are expressed relative to the dexterous hand wrist position to reduce spatial complexity while preserving the direction of gravity.

We model the task as a Markov Decision Process (MDP) $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma \rangle$. The action for each hand at time t comprises target joint positions $a_q^t \in \mathbb{R}^K$ for proportional-derivative (PD) control of the fingers, and a 6-DoF wrench $a_w^t \in \mathbb{R}^6$ applied directly to the robotic wrist.

3.2 Two-Stage Policy Architecture

Our approach builds on the ManipTrans framework [1], which separates the manipulation transfer problem into two sequential stages: a pre-trained trajectory imitation module and a residual correction module. We adopt this architecture and describe it here for completeness. The difference lies in how we use it: instead of fitting a separate policy to each individual demonstration, we train a single shared policy across a set of demonstrations, as shown in Figure 1.

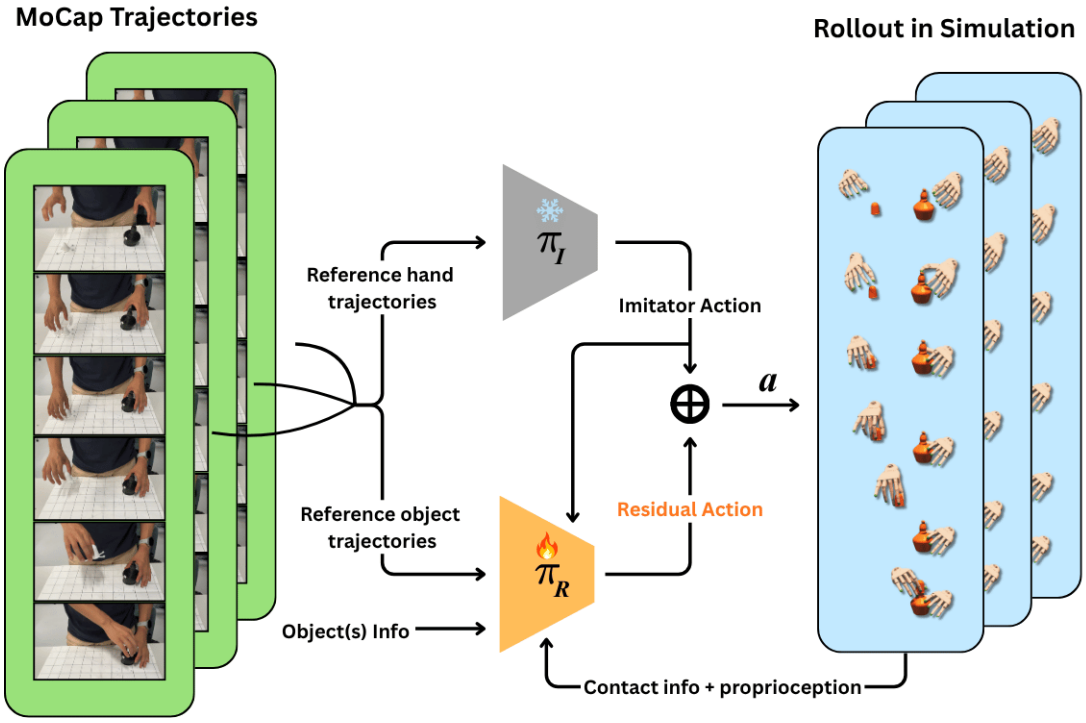

Figure 1: Our system architecture. We use a hand motion imitation model pre-trained on large-scale human demonstrations, then train a residual policy on top of it to adapt to a multi-demonstration task.

Stage 1 — Hand Trajectory Imitator Policy The imitator π_I is a pre-trained generalist policy that drives each robotic hand to track the reference human hand trajectory without any object interaction. Its state at time t is

$$s_I^t = \{\tau_h^t,\ s_{\text{prop}}^t\}, \qquad s_{\text{prop}}^t = \{q_d^t,\ \dot{q}_d^t,\ w_d^t,\ \dot{w}_d^t\}, \tag{1}$$

where q_d^t and w_d^t denote the finger joint angles and wrist pose respectively, together with their velocities. At each step the imitator samples an action $a_I^t \sim \pi_I(a^t \mid s_I^t,\ a^{t-1})$.

Stage 2 — Residual Policy. A residual module π_R refines the imitator's output to satisfy the task-specific physical constraints arising from object interaction. Its state $s_R^t = s_I^t \cup s_{\text{interact}}^t$ augments the imitation state with object-related information

$$s_{\text{interact}}^t = \{\tau_o^t,\ p_{\hat{o}}^t,\ \dot{p}_{\hat{o}}^t,\ m_{\hat{o}},\ G_{\hat{o}},\ \text{BPS}(\hat{o}),\ D(j_d^t, p_{\hat{o}}^t),\ C^t\}, \tag{2}$$

where $p_{\hat{o}}^t$ and $\dot{p}_{\hat{o}}^t$ are the simulated object position (relative to the wrist) and velocity, $m_{\hat{o}}$ and $G_{\hat{o}}$ are the object center of mass and gravitational force vector, $\text{BPS}(\hat{o})$ is a Basis Point Set encoding [21] of the object shape, $D(j_d^t, p_{\hat{o}}^t)$ is the squared Euclidean distance from each hand keypoint to the object, and C^t is the contact force at each fingertip obtained from the simulator.

At each step, the residual correction $\Delta a_R^t \sim \pi_R(\Delta a^t \mid s_R^t,\ a_I^t,\ a^{t-1})$ is added element-wise to the imitator action, giving $a^t = a_I^t + \Delta a_R^t$, clipped to the joint limits of the hand. Because the imitator already provides a reasonable motion prior, the residual corrections are expected to be near zero at the start of training. We follow [1] in initializing the residual module with a zero-mean Gaussian weight distribution and applying a linear warm-up schedule to its training, which prevents collapse and accelerates convergence. During training, the imitator weights are frozen; only the residual policy is updated.

3.3 Reward Design

Imitator reward. We use the pre-trained imitator provided by ManipTrans [1] directly and do not retrain it. For reference, its reward combines wrist tracking, finger keypoint tracking, and motion smoothness:

$$r_I^t = w_{\text{wrist}} \cdot r_{\text{wrist}}^t + w_{\text{finger}} \cdot r_{\text{finger}}^t + w_{\text{smooth}} \cdot r_{\text{smooth}}^t. \tag{3}$$

The finger imitation term takes the form

$$r_{\text{finger}}^t = \sum_{f=1}^{F} w_f \cdot \exp\Big(-\lambda_f \left\| j_{d_f}^t - j_{h_f}^t \right\|_2^2\Big), \tag{4}$$

where $F = 21$ keypoints are selected on each robotic hand to correspond to the MANO model [22], and weights w_f and decay rates λ_f are set to emphasize the fingertips of the thumb, index, and middle fingers.

Residual reward. For the residual stage we extend the imitation reward with two object-centric terms:

$$r_R^t = r_I^t + w_{\text{object}} \cdot r_{\text{object}}^t + w_{\text{contact}} \cdot r_{\text{contact}}^t. \tag{5}$$

The *object following* reward r_{object}^t penalizes positional and velocity deviation of the simulated object from its reference trajectory. The *contact* reward r_{contact}^t encourages appropriate fingertip forces whenever the reference hand-to-object distance falls below a threshold ξ_c:

$$r_{\text{contact}}^t = w_c \cdot \exp\left(\frac{-\lambda_c}{\sum_{f=1}^{F} C_{d_f}^t \cdot \mathbf{1}\Big(D\Big(j_{h_f}^t,\ p_o^t \cdot o\Big) < \xi_c\Big)} \right), \tag{6}$$

where $j_{h_f}^t$ is the position of the f-th human fingertip at time t and p_o^t is the reference pose of object o. The product $p_o^t \cdot o$ denotes the object mesh o rigidly transformed by that pose, i.e.

the object surface as it is posed at time t, and $D(\cdot, \cdot)$ is the minimum Euclidean distance from a point to that surface. The indicator $\mathbf{1}(\cdot)$ is therefore one exactly when the reference fingertip lies within ξ_c of the object surface, so only fingertips that the demonstration places in contact contribute to the sum. $C^t_{d_f}$ is the contact force measured at the corresponding fingertip of the dexterous hand, w_c scales the term and λ_c sets its decay rate.

This reward formulation deliberately avoids task-specific engineering, keeping the signal general across demonstrations — a property that is central to our multi-demonstration generalization objective.

4 Experiments

We devise a series of experiments to validate the methodology. In this section, we describe the experimental setup (Section 4.1), report results of the model evaluated through online (Section 5.2) and offline methods (Section 5.1), and also perform ablation studies (Section 5.1.1). We use two tasks: alcohol bottle capping and brush flipping, and evaluate the full residual-imitation model against dex-retargeting [11] (a classical optimization-based method) and pure imitation model-based retargeting, using the frozen pre-trained imitator from ManipTrans [1].

4.1 Experimental Setup

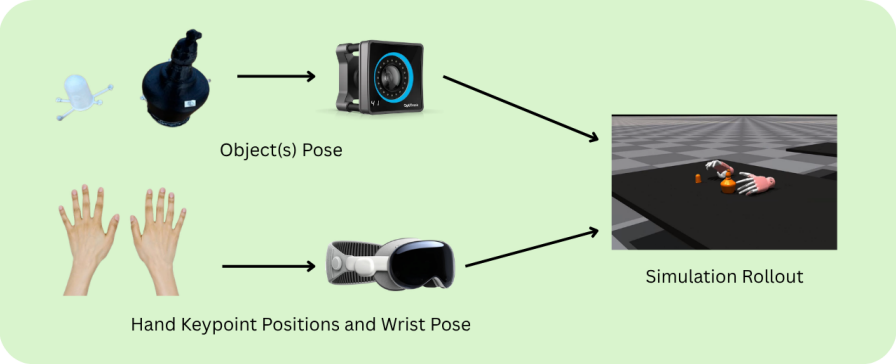

Figure 2: Overview of the data collection pipeline, showing how hand and object data flows from the Apple Vision Pro and OptiTrack systems through to the Isaac Gym simulator.

As seen in Figure 2, to collect training data or perform teleoperation, we use two systems, the Apple Vision Pro (AVP) and the OptiTrack. The simulator and data streaming run at 60 Hz. Using the AVP SDK and VisionProTeleop [23], we collect the 21 keypoints and wrist pose per hand, and with the OptiTrack, we record the object poses. The objects used are an alcohol burner and its cap for the capping task, and a brush and a holder for the brush-flipping task. The burner and cap were sourced from the Oakink2 [19] dataset's mesh files and modified to include the OptiTrack markers. The brush and holder are modelled as primitives rather than taken from a dataset, dimensioned after everyday objects: the brush is a $2.5 \times 20 \times 1.5\,\text{cm}$ rectangular prism sized after a toothbrush, and the holder a $4 \times 5 \times 4\,\text{cm}$ cup sized after a minimalist brush holder. On all objects the markers were placed in a manner that minimally affects the user's manipulation experience.

Since the two systems have different coordinate frames, we perform a calibration to map the two into a single frame by using a tracker on the tip of the index finger of the right hand. By moving the hand around in the workspace of the AVP and the OptiTrack we have a set of two points

that we use the Umeyama [6] algorithm to map from the AVP to OptiTrack. The mapping is not perfect, which may be attributable to the image-based machine-learning techniques the Apple Vision Pro uses for finger and wrist pose estimation, and the tracker placement on the fingertip potentially varying slightly in position between sessions. We determined that the speed of the movements within the space had a positive linear correlation with the remapping error. Moving at a reasonable manipulation speed of around 200-400 mm/s, the RMSE between the systems was around 8-12 mm.

Teleoperation Architecture Pipeline Figure 3 shows the live capture and teleoperation loop. The AVP and OptiTrack stream data at 60Hz, over WiFi and Ethernet respectively. The laptop fuses the two into a single frame — the Umeyama AVP→OptiTrack calibration places hands and objects in a common Motive frame, stamps it with a sequence number and capture time, and publishes it as `msgpack` over a ZeroMQ PUB socket at 60 Hz. The desktop subscribes and runs ManipTrans in Isaac Gym at roughly 60 Hz. The desktop's rendered viewer returns to the laptop over VNC at 15 Hz, closing the loop for the operator.

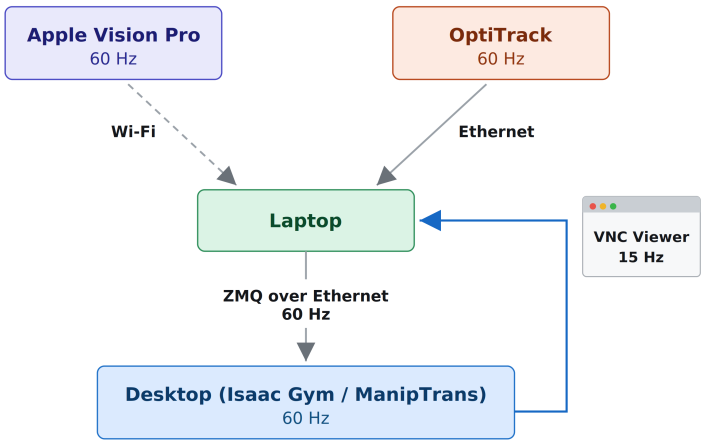

Figure 3: Live capture and teleoperation pipeline. Annotated values are the measured rate of each stage in Hz.

We train the residual module R with the actor-critic PPO algorithm [24], optimized with Adam [25]; the full set of network and PPO hyperparameters is given in Table 9. All simulation is carried out in Isaac Gym [26]. Training is performed on an HPC cluster equipped with NVIDIA V100 and A100 GPUs, while evaluation, visualization, and teleoperation are run on a single NVIDIA RTX 4090 GPU.

4.2 Data collection

Demonstrations Both tasks embody real-world manipulation scenarios: the first involves capping an alcohol burner bottle of the kind found in a laboratory, and the second involves reorienting a brush. Both are illustrated in Figure 4, and the physical props used for each are shown in Figure 5.

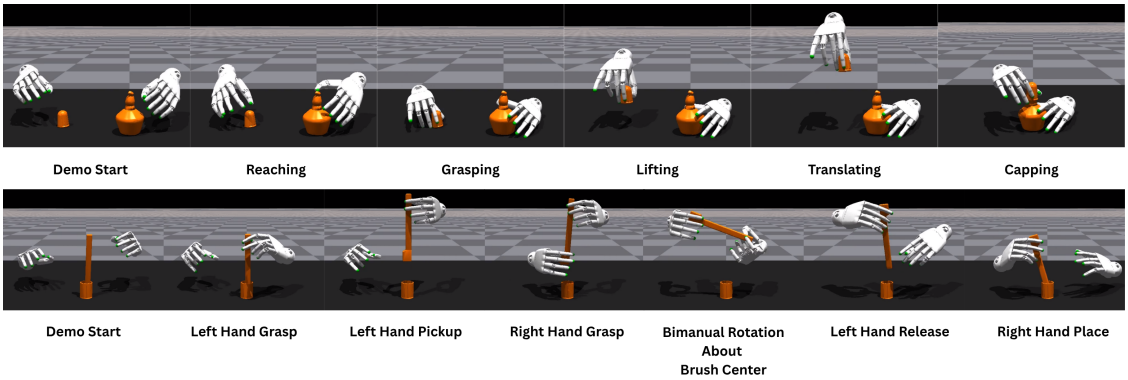

Figure 4: The task decomposition of the bottle-capping and brush-flipping manipulation tasks.

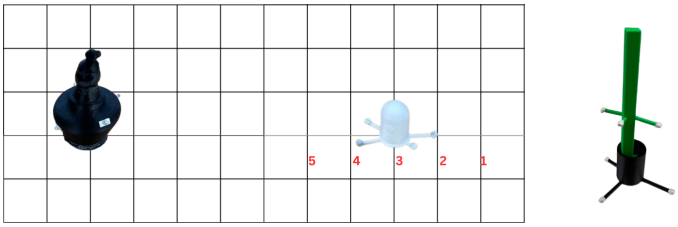

Figure 5: The props used for data collection for the bottle-capping and brush-flipping demonstrations.

For the bottle-capping demonstrations, we use a grid-based system to enable repeatable, generalizable data collection. We collect trajectories from 5 distances away from the bottle body where the left hand holds the bottle body and the right hand grasps the cap. As seen in Figure 4, both hands start above the objects in a neutral position and reach towards them. The right hand picks up the cap and moves it towards the body, while the left hand stabilizes the bottle, and the demo terminates when the cap is seated on top of the bottle.

For the brush demonstration, we collect data such that the initial orientation of the brush is constant, and the trajectory is changed by lifting the brush to different heights and using different rotation speeds. To encourage generalization we rotate the brush at different velocities. Similar to the bottle capping task, the hands start in a neutral palm facedown pose, the left hand grasps the end of the brush, lifts it up, then the right hand grasps the bristled end of the brush and both hands rotate the brush about the center point. Then the left hand releases the brush, and the right hand places it back into the cup.

Data Augmentation To increase model generalizability, we augment the bottle-capping demonstrations by applying affine transformations to the hand and/or objects across the entire trajectory. The transformations include rotations of the hand and bottle trajectories, which are applied using a matrix operation as seen in:

$$\hat{A}_{\text{pose}}[\tau_k] = T_{\text{trans}} \cdot A_{\text{pose}}[\tau_k] \tag{7}$$

Here, $A_{\text{pose}}[\tau_k]$ refers to the pose of either the hand or the object at the k-th frame of trajectory τ. A rigid transformation $T_{\text{trans}} \in SE(3)$ is applied to each such pose, yielding the augmented pose $\hat{A}_{\text{pose}}[\tau_k]$.

4.3 Model Performance

We perform a series of tests to motivate the need for training a more comprehensive model that can generalize to different tasks and a more informed algorithm than pure kinematic retargeting. Tests were performed both offline, by playing back demonstrations, and online, using a live teleoperation setup. To evaluate the online performance we used qualitative metrics, and offline, we used both quantitative and qualitative metrics. For qualitative evaluation, we ensured that the hand was manipulating the object with the same fingers used as the operator or demo. In some cases, the model learned to pinch the cap between the index and middle finger rather than between the thumb and index finger. For quantitative evaluation, we roll out each demonstration 128 times on a policy and record the success rate, which is described below.

4.3.1 Evaluation Metrics

For offline demonstrations, we evaluate all conditions using three quantitative metrics following ManipTrans [1], averaged over all successful rollouts with bimanual metrics computed per hand and averaged across both hands: **object translation error** $E_t = \frac{1}{T}\sum_{t=1}^{T} \|p_{\hat{o}}^{\mathrm{tsl},t} - p_o^{\mathrm{tsl},t}\|_2$ (cm), **object rotation error** $E_r = \frac{1}{T}\sum_{t=1}^{T} \|\log(R_{\hat{o}}^{\mathrm{rot},t}(R_o^{\mathrm{rot},t})^{-1})\|_2$ (degrees), and **mean per-fingertip error** $E_{ft} = \frac{1}{T\cdot M}\sum_{t=1}^{T}\sum_{f=1}^{M} \|t_{d_f}^t - t_{h_f}^t\|_2$ (cm) over M=2 fingertip joints (index and thumb) because they are most used for the manipulation tasks. This departs from ManipTrans [1], which averages the same error over the full fingertip set; E_{ft} and the success rates derived from it are therefore not directly comparable with the values reported there. A rollout is successful if all three errors remain below the thresholds $E_r < 45°$, $E_t < 3\,\mathrm{cm}$, and $E_{ft} < 6\,\mathrm{cm}$; for bimanual tasks both hands must satisfy all conditions simultaneously. The success rate (SR) is the fraction of successful rollouts per demonstration.

For online demonstrations, task success is evaluated qualitatively by visual inspection, with each task decomposed into subtasks and completion assessed at the phase level.

5 Experimental Results

5.1 Offline Demonstration

Cross-validation protocol. The 50 reach demonstrations, 10 at each of five reach distances, are partitioned into five folds by a deterministic stratified split: demonstrations are grouped by distance and dealt two per distance to each fold, so every fold holds out 10 demonstrations spanning all five distances and trains on the remaining 40. For each held-out demonstration we score 128 completed episodes, i.e. 1,280 episodes per fold. An episode counts as a success if it reaches the final demonstration frame with both hands never exceeding the evaluation tolerances — 6 cm thumb- and index-tip tracking error, 3 cm object position error and 45° object orientation error. The success rate of a demonstration is the fraction of its scored episodes that succeed; each cell of Figure 6 averages the two demonstrations that fall in that (fold, distance) pair, with marginal means over folds and over distances.

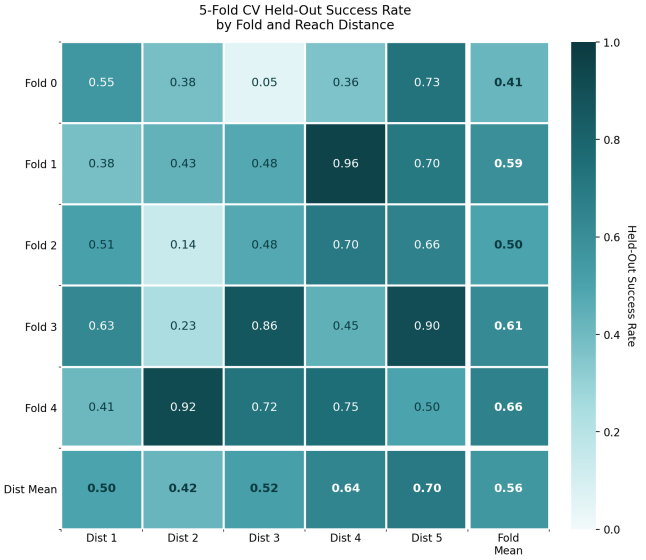

Figure 6: K-fold matrix results describing the success rate per fold and per demonstration distance.

We notice that some of the demonstrations have very low success rates, and in many of the cases, this is due to a failure of the initial grasp, or a grasp that is unstable, so the cap slips out from between the fingertips. Notably, there seems to be a trend of improvement as the distance decreases. This could be attributed to the hands being closer to the AVP, which may lead to more accurate retargeting.

5.1.1 Bottle Capping

We compare the results of the offline rollout to the kinematic retargeting imitation only policy and dex-retargeting [11]. To understand overall performance at a granular level, we decompose the task by hand and phase. We evaluate the phase-level success counts over 10 zero-shot demonstrations for both the bottle-capping and the brush-flipping demonstrations. Throughout, *zero-shot* denotes demonstrations that were held out from the policy's training set: the policy is evaluated on them without having been trained on them.

The results in Table 1 separate the three methods by phase. All three complete the approach identically — the left hand reaches and stabilizes the bottle and the right hand reaches the cap in 10/10 demonstrations for every method — so the comparison is decided entirely in the right hand's contact phases. The imitator fails at the first of them: it does not grasp the cap in any of the 10 demonstrations. This is expected: the imitator tracks the kinematic reference trajectory but has no mechanism to respond to contact forces or object resistance. When the right hand closes around the cap, the fingers reach the reference joint angles but without contact-aware correction, the fingertips fail to generate sufficient grip force to lift the cap.

The residual policy recovers this gap by conditioning on fingertip contact forces and hand-to-object distances, allowing it to actively adjust joint targets as the fingers approach and close around the cap. This is reflected in the results: all 10 demonstrations grasp the cap, and 8/10 go on to lift and translate it. The single additional failure at seating (7/10 vs. 8/10) is consistent with the task's most contact-critical moment — placing the cap requires simultaneously maintaining grip and applying downward force, so a single lost contact at this stage results in failure.

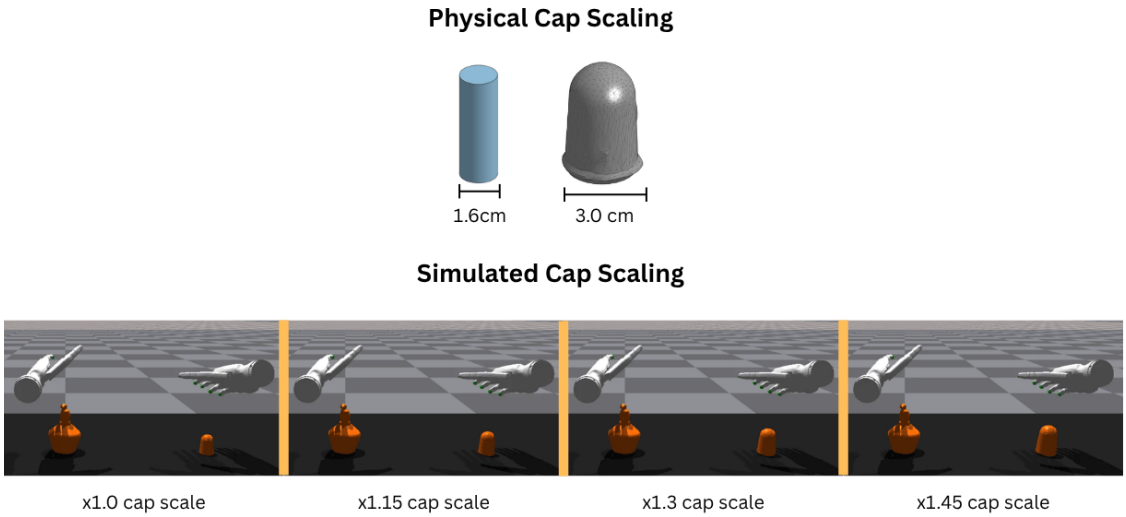

Figure 7: Physical cap ablation: outer diameter reduced by 1.4 cm, and simulated cap ablation by scaling the mesh size.

The dex-retargeting baseline outperforms the residual policy over the transport phases. It matches the residual policy at the grasp (10/10 for both) and exceeds it at lifting and translating the cap (10/10 against 8/10). Because it solves directly for a fingertip configuration matching the operator's hand geometry, it produces a repeatable and geometrically precise pinch, and it lifts and transports the cap in all 10 demonstrations.

The pose of the cap within that pinch, however, is not controlled. Without observation of fingertip contact, the thumb and index finger do not close on the cap simultaneously, and the digit that contacts first rotates the cap between the fingertips before the grip settles. The resulting pinch is stable, which accounts for the 10/10 lift and translate counts, but it is stable about an orientation that differs from the one at which the cap was grasped. The method receives no signal that this rotation has occurred and provides no mechanism to correct it, so the cap reaches the bottle at an uncontrolled angle. Seating requires the cap axis to be aligned with the bottle opening within a small tolerance, and the retargeter satisfies this condition in none of the 10 demonstrations. The residual policy, conditioned on fingertip contact forces and hand-to-object distances, preserves the grasped orientation and seats the cap in 7/10.

Hand	Phase	dex-retargeting	Imitator	Residual (ours)
Left	Reach bottle	10 / 10	10 / 10	10 / 10
	Stabilize bottle	10 / 10	10 / 10	10 / 10
Right	Reach cap	10 / 10	10 / 10	10 / 10
	Grasp cap	10 / 10	0 / 10	10 / 10
	Lift cap	10 / 10	0 / 10	8 / 10
	Translate cap	10 / 10	0 / 10	8 / 10
	Seat cap	0 / 10	0 / 10	7 / 10

Table 1: Phase-level success counts over 10 zero-shot demonstrations, broken down by hand and task phase.

Cap-Scaling Ablations To motivate the need to assist the imitation model with contact force, we performed an ablation study that injects contact force by scaling the cap using both physical and software methods, illustrated in Figure 7. To improve the pinching capability of the

imitator, we designed a thinner cap with an outer diameter of 1.6 cm, compared to roughly 3.0 cm for the original. Evaluating the manipulation task with the imitator over 13 demonstrations, we observe improved performance, demonstrating that object-finger interactions are crucial for manipulation tasks.

We combine this with software-based ablations by scaling the thin cap's mesh by up to 1.45×; results for the right hand are reported in Table 2.

Model	**Phase**	**×1.00**	**×1.15**	**×1.30**	**×1.45**
dex-retargeting	Reach cap	13 / 13	13 / 13	13 / 13	13 / 13
	Grasp cap	11 / 13	12 / 13	13 / 13	12 / 13
	Lift cap	10 / 13	12 / 13	13 / 13	11 / 13
	Translate cap	9 / 13	11 / 13	12 / 13	10 / 13
	Seat cap	0 / 13	1 / 13	1 / 13	1 / 13
Pre-Trained Imitator	Reach cap	13 / 13	13 / 13	13 / 13	13 / 13
	Grasp cap	3 / 13	8 / 13	9 / 13	8 / 13
	Lift cap	3 / 13	5 / 13	6 / 13	5 / 13
	Translate cap	3 / 13	4 / 13	5 / 13	4 / 13
	Seat cap	1 / 13	3 / 13	4 / 13	3 / 13
Fine-Tuned Residual (ours)	Reach cap	13 / 13	13 / 13	13 / 13	13 / 13
	Grasp cap	13 / 13	13 / 13	13 / 13	12 / 13
	Lift cap	12 / 13	13 / 13	13 / 13	12 / 13
	Translate cap	11 / 13	12 / 13	12 / 13	11 / 13
	Seat cap	10 / 13	11 / 13	12 / 13	10 / 13

Table 2: Phase-level success counts over 13 demonstrations for each model across four cap diameter scales. All columns use the thin 1.6 cm physical cap and variable levels of mesh scaling.

What is observed is that the thinner cap on its own improves the performance of the pre-trained imitator model, as there is one demonstration that can seat the cap. However, we can see a large improvement with the thinner cap and the scaled model inside the simulation. The largest jump is between x1.0 and x1.15 where the seating of the cap for the imitator model goes from 1 to 3. Overall, scaling the cap benefits the imitator model the most, which makes sense, as it is the least accurate of the three methods at kinematic retargeting. The imitator has no observation of the object and therefore closes its fingers to the same joint targets regardless of cap size, so enlarging the cap mesh means those fingers now pinch past the object's original diameter. This deeper interpenetration lets the simulator apply a correspondingly larger contact force at the fingertips, in effect supplying grip force through the geometry that the imitator itself never learned to apply. This peak-at-×1.30, decline-at-×1.45 pattern is not specific to the imitator: it holds for all three methods across every contact-critical phase, including the residual policy's own grasping, lifting, translation and seating counts. The only two exceptions are the reach phase, which is already saturated at 13/13 for every method at every scale and therefore cannot show the pattern, and dex-retargeting's seating count, which jumps from 0 to 1 at ×1.15 and then plateaus rather than declining — consistent with that method's seating failure being a question of grasp orientation rather than grip force, which cap size does not address. The across-the-board decline at ×1.45 likely has a simpler cause than distribution shift alone: at that scale the cap is large enough to interfere with manipulation directly, leaving the fingers too little clearance to close around it with the same grasp the demonstration was recorded with, which would depress every method's numbers regardless of how well each tracks or corrects the

reference trajectory.

5.1.2 Brush Flipping

The results in Table 3 reveal a sharp contrast between the two learned policies. The imitator completes the initial left-hand reach in all ten demonstrations but fails at every subsequent phase, never once achieving a stable grasp. The failure mode is consistent across all trials: the fingertips approach the brush but do not close around it. We attribute this to two factors. First, the retargeted fingertip positions carry a residual error relative to the ground-truth human fingertip trajectories, and because grasping is highly sensitive to millimeter-scale fingertip placement, even a small offset is sufficient to leave the digits short of the object surface. Second, the imitator is trained purely on kinematic targets and receives no contact or force feedback, so it has no signal with which to detect that a grasp has not been established and no mechanism to correct the closure online. The policy therefore executes an open-loop approximation of the demonstrated motion and proceeds as if the object were held.

The residual policy completes the full sequence in five of ten demonstrations, with the remaining failures distributed across the later phases. The failures are no longer concentrated at the grasp: the corrective term is sufficient to close the fingers around the brush in nine of ten trials, and the remaining errors are distributed across the later, more constrained phases. Two distinct failure mechanisms account for the losses. The first is grasp quality. One demonstration fails outright at the left-hand lift, and a second establishes a poor grasp that only manifests later, during the right-hand reach, when the brush is no longer held securely enough for the second hand to engage it. The second and dominant mechanism is the handover itself. In two demonstrations the left hand fails to release cleanly after the bimanual rotation and remains caught on the brush, blocking the transfer. Notably, these two demonstrations have a grasp that is lower on the brush, which causes the left hand release to fail. A further demonstration completes the release but strikes the rim of the cup during seating, so the brush is never inserted. The bimanual rotation phase itself incurs no additional failures, indicating that once a secure two-handed configuration is reached, the reorientation is reliable. The bottleneck lies instead at the transitions into and out of that configuration, where fine control of contact release and terminal placement is required — precisely the regime in which the residual correction, trained without explicit contact feedback, has the least to work with.

The dex-retargeting baseline shares the imitator's fundamental limitation at the grasp: the vector-based objective optimizes for a different quantity than fingertip alignment, so the left hand frequently collides with or fails to contact the brush rather than closing around it. Unlike the imitator, this failure is recoverable in a live teleoperation setting where the operator can compensate online, but in the offline rollout evaluated here there is no such corrective mechanism, and the method does not progress beyond the initial reach.

Phase	**Hand**	**dex-retargeting**	**Imitator**	**Residual (ours)**
Reach	Left	10 / 10	10 / 10	10 / 10
Lift	Left	0 / 10	0 / 10	9 / 10
Reach	Right	0 / 10	0 / 10	8 / 10
Rotation	Bimanual	0 / 10	0 / 10	8 / 10
Release	Left	0 / 10	0 / 10	6 / 10
Seat	Right	0 / 10	0 / 10	5 / 10

Table 3: Phase-level success counts over 10 zero-shot demonstrations of the brush-flipping task, broken down by task phase and hand.

5.2 Online Demonstration

We further develop the need of the generalized model through testing in teleoperation. We collect 10 consecutive trajectories from each distance, for a total of 50 demonstrations to generate the metrics below for both tasks. The success metric is fully determined qualitatively by evaluating the progress through the task.

5.2.1 Bottle Capping Teleoperation

Hand	Phase	dex-retargeting	Imitator	Residual (ours)
Left	Reach bottle	10 / 10	10 / 10	10 / 10
	Stabilize bottle	10 / 10	10 / 10	10 / 10
Right	Reach cap	10 / 10	10 / 10	10 / 10
	Grasp cap	7 / 10	10 / 10	9 / 10
	Lift cap	7 / 10	1 / 10	9 / 10
	Translate cap	6 / 10	0 / 10	8 / 10
	Seat cap	0 / 10	0 / 10	5 / 10

Table 4: Phase-level success counts over 10 consecutive teleoperation demonstrations at demonstration distance 5, broken down by hand and task phase.

Qualitatively, the manipulation of the cap using the residual model requires the least amount of user feedback. The grasping and capping — the most interaction-heavy parts of the trajectory — were assisted by the model. However, this model is unstable, and going out of distribution resulted in catastrophic spinning of the hands at high velocity. The dex-retargeting approach was the second most difficult to operate. While grasping succeeded in 7 of 10 trials, the cap would frequently rotate during the pinching grasp due to the sensitivity of the method to finger closing coordination. Because the operator hand pose is mapped directly to robot joint targets without contact-aware correction, the timing at which individual fingertips contact the cap surface is determined entirely by the operator's motion. If the index finger and thumb do not close at the same rate, one fingertip contacts the cap before the other, generating an asymmetric force that causes the cap to tilt or rotate rather than be gripped stably. This forces the operator to compensate by rotating the entire wrist, making seating the cap nearly impossible. Attempting to mitigate this with a three-finger grasp did not improve reliability; introducing a third finger increases the number of pairwise contact timing differences from one to three, making synchronized contact even harder to achieve in practice. This suggests that pure kinematic retargeting is fundamentally limited for precision grasping tasks, as it places the full burden of contact coordination on the operator rather than the policy.

Finally, the ManipTrans imitation-only model demonstrates the weakest performance during teleoperation: although the fingers close around the cap in all ten trials, the resulting grasp is secure enough to lift it in only one. The fundamental limitation is that the imitator reproduces the reference trajectory kinematically but produces only coarse positional targets, without the fine-grained fingertip precision required for contact-critical manipulation. Those targets are sufficient to bring the fingers into the vicinity of the cap and close them around it, but not precise enough to seat the fingertips on its surface with enough force to retain it, so the cap slips as soon as the hand lifts. This mirrors the offline results in Table 1, where the imitator never lifts or translates the cap, and confirms that coarse kinematic imitation alone is insufficient for precision grasping tasks.

Failure Mode	Dist. 1	Dist. 2	Dist. 3	Dist. 4	Dist. 5
Missed Grasp	0	2	1	1	1
Missed Translate	0	0	2	0	1
Missed Cap	1	2	3	2	3
Failed Lift	1	0	0	0	0
Total	2	4	6	3	5

Table 5: Breakdown of residual-based teleoperation failure modes for the bottle-capping task across the five demonstration distances, aggregated over ten trials per distance.

Quantitatively, we observe that demonstration distance itself does not correlate with teleoperation failure rate; rather, performance is primarily determined by task phase. The initial grasp and final seating phases account for the largest share of failures, with the residual policy dropping from fifty out of fifty successes at the reach phase to thirty out of fifty by the seating phase. This indicates that the precision required to align fingers to the cap's radius during grasping, and to align the cap's descent point with the bottle opening during seating, are the two dominant bottlenecks in the teleoperation pipeline, rather than any degradation from operator distance or trajectory novelty.

Residual Model Performance Two qualitative failure modes were observed during live teleoperation, both consistent with distribution shift. First, when the operator deliberately slows or pauses their movements, for instance, while visually cross-referencing the simulated hand against the real one, the resulting frame-to-frame deltas fall outside the velocity distribution the policy was trained on, pushing the observation out of distribution and increasing the likelihood of failure. Second, the policy becomes noticeably less stable during the capping phase at the end of the episode. We attribute this to occlusion: as the two hands converge on the burner they increasingly obscure one another and the object, degrading the headset's keypoint estimates at precisely the moment the tracking targets must be most accurate. Together these suggest that the policy's robustness to variation in demonstrated trajectories does not extend to variation in motion speed or to degraded target quality, and that operator guidance, per-step target noise, and training-time augmentation across a wider range of speeds would improve reliability in practice.

5.2.2 Brush-Flipping Teleoperation

Phase	Hand	dex-retargeting	Imitator	Residual (ours)
Reach	Left	8 / 10	10 / 10	10 / 10
Lift	Left	7 / 10	0 / 10	10 / 10
Reach	Right	5 / 10	0 / 10	9 / 10
Rotation	Bimanual	0 / 10	0 / 10	4 / 10
Release	Left	0 / 10	0 / 10	3 / 10
Seat	Right	0 / 10	0 / 10	0 / 10

Table 6: Phase-level success counts over 10 teleop demonstrations of the brush-flipping task, broken down by task phase and hand.

For the brush-flipping task, the imitator model proves ineffective throughout, as the mapping between the demonstrated hand pose and the robot's joint targets is not precise enough to establish any manipulation capability. In every trial, the fingertips approach the brush but fail to close around it, so no phase beyond the initial reach is ever completed. The dex-retargeting approach performs better, reaching and lifting the brush in 8 and 7 out of 10 trials respectively, but fails to complete the rotation in any trial. The grasping and rotation are unintuitive to operate under this method, since the retargeter optimizes a separate objective from the manipulation goal directly, requiring constant operator correction that slows execution considerably. The residual model is the most capable of the three, successfully reaching and lifting in all trials and completing the rotation in 4 out of 10, but no method succeeds at seating the brush. A key difficulty is that the rotation phase requires the fingers to maintain contact with the brush while simultaneously allowing it to rotate between them — a regime in which simulated friction and contact dynamics may not accurately reflect real-world behavior. The longer horizon of this task relative to bottle capping compounds these difficulties, as errors accumulate across phases and leave less margin for recovery at the terminal seating step.

6 Limitations and Future Work

The shared residual policy does transfer to held-out demonstrations, but training one policy across many demonstrations at once introduces two costs. First, distributing a fixed number of parallel simulation environments across many demonstrations reduces the per-demonstration sample count, which can destabilize policy optimization. Second, and more fundamentally, demonstrations of differing object geometries, contact configurations, or motion trajectories may induce conflicting policy gradients—corrections beneficial for one demonstration may be detrimental to another—causing the shared policy to converge to a compromise that satisfies one demonstration rather than another. Additionally, there are limitations to learning, as there is noise from the AVP and OptiTrack systems; specifically, the Apple Vision struggles when there are occlusions of the hand due to object interactions. A further limitation stems from inaccuracies in the object meshes and small errors in their estimated poses. Such mismatches caused failures in simulation — for instance, collisions between meshes — despite the corresponding real-world configuration being properly aligned. Finally, two aspects of the evaluation bound the scope of these results. All demonstrations and teleoperation trials were recorded from a single operator, so although the policy is exposed to a range of demonstration styles, its robustness to inter-operator variation in hand size, grasp preference, and motion timing remains untested. The evaluation is also carried out entirely in simulation — no policy was deployed on physical robotic hands — so the sim-to-real gap in contact dynamics, friction, and actuator response is not assessed here, and transfer to real hardware remains the natural next step.

7 Conclusion

We introduce a bimanual teleoperation system that takes a concrete step toward generalizable dexterous manipulation. Built on our proposed multi-demonstration residual policy with cross-system spatial calibration, our approach learns contact-aware manipulation without task-specific reward engineering and operates on unseen operator trajectories without retraining. Across two long-horizon bimanual tasks it is the only approach evaluated that completes the full manipulation sequence end to end in offline rollout, where both the pre-trained imitator and dex-retargeting fail at contact-critical phases. Under live teleoperation the same policy completes the capping task, while the longer-horizon brush task is not seated by any of the

three methods. Furthermore, the same hardware pipeline that collects training demonstrations drives live teleoperation without modification, establishing a scalable foundation for dexterous bimanual data collection and a viable path toward robust human-level robotic manipulation.

Acknowledgments

The author would like to thank Cheng Pan and Prof. Josie Hughes for their guidance and support throughout this project. Parts of the code and report were developed with the assistance of Claude (Anthropic), used for code generation, debugging, and drafting of written content. All outputs were reviewed, verified, and edited by the author.

8 References

[1] Kailin Li et al. "ManipTrans: Efficient Dexterous Bimanual Manipulation Transfer via Residual Learning". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2025. arXiv: 2503.21860 [cs.RO]. URL: https://arxiv.org/abs/2503.21860.

[2] Mandi Zhao et al. "DexMachina: Functional Retargeting for Bimanual Dexterous Manipulation". In: *arXiv preprint arXiv:2505.24853* (2025). arXiv: 2505.24853 [cs.RO]. URL: https://arxiv.org/abs/2505.24853.

[3] Xueyi Liu et al. *DexTrack: Towards Generalizable Neural Tracking Control for Dexterous Manipulation from Human References*. 2025. arXiv: 2502.09614 [cs.RO].

[4] Xueyi Liu et al. "Parameterized Quasi-Physical Simulators for Dexterous Manipulations Transfer". In: *European Conference on Computer Vision (ECCV)*. 2024.

[5] Sirui Xu et al. "Dexplore: Scalable Neural Control for Dexterous Manipulation from Reference-Scoped Exploration". In: *Conference on Robot Learning (CoRL)*. 2025.

[6] S. Umeyama. "Least-Squares Estimation of Transformation Parameters Between Two Point Patterns". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 13.4 (1991), pp. 376–380. DOI: 10.1109/34.88573.

[7] Marcin Andrychowicz et al. "Learning Dexterous In-Hand Manipulation". In: *The International Journal of Robotics Research* 39.1 (2020), pp. 3–20.

[8] Ankur Handa et al. "DeXtreme: Transfer of Agile In-Hand Manipulation from Simulation to Reality". In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2023, pp. 5977–5984.

[9] Xue Bin Peng et al. "DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills". In: *ACM Transactions on Graphics (TOG)* 37.4 (2018), pp. 1–14.

[10] Ankur Handa et al. "DexPilot: Vision-Based Teleoperation of Dexterous Robotic Hand-Arm System". In: *IEEE International Conference on Robotics and Automation (ICRA)*. May 2020, pp. 9164–9170.

[11] Yuzhe Qin et al. "AnyTeleop: A General Vision-Based Dexterous Robot Arm-Hand Teleoperation System". In: *Robotics: Science and Systems (RSS)*. 2023.

[12] Aadhithya Iyer et al. *OPEN TEACH: A Versatile Teleoperation System for Robotic Manipulation*. 2024. arXiv: 2403.07870 [cs.RO].

[13] Runyu Ding et al. "Bunny-VisionPro: Real-Time Bimanual Dexterous Teleoperation for Imitation Learning". In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2025, pp. 12248–12255.

[14] Xuxin Cheng et al. "Open-TeleVision: Teleoperation with Immersive Active Visual Feedback". In: *Conference on Robot Learning (CoRL)* (2024). arXiv: 2407.01512.

[15] Alejandro Posadas-Nava, Alejandro Carrasco, and Richard Linares. "BEAVR: Bimanual, multi-Embodiment, Accessible, Virtual Reality Teleoperation System for Robots". In: *2025 7th International Conference on Control and Robotics (ICCR)*. IEEE, Dec. 2025, pp. 283–291. DOI: 10.1109/iccr67607.2025.11372114. URL: http://dx.doi.org/10.1109/ICCR67607.2025.11372114.

[16] Puhao Li et al. *Towards Human-level Dexterous Teleoperation*. 2026. arXiv: 2607.11481 [cs.RO]. URL: https://arxiv.org/abs/2607.11481.

[17] Jielin Wu et al. "TopoRetarget: Interaction-Preserving Retargeting for Dexterous Manipulation". In: *arXiv preprint arXiv:2606.16272* (2026).

[18] Chen Wang et al. *DexCap: Scalable and Portable MoCap Data Collection System for Dexterous Manipulation*. 2024. arXiv: 2403.07788 [cs.RO].

[19] Xinyu Zhan et al. "OakInk2: A Dataset of Bimanual Hands-Object Manipulation in Complex Task Completion". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024, pp. 445–456. arXiv: 2403.19417 [cs.CV]. URL: https://arxiv.org/abs/2403.19417.

[20] Zicong Fan et al. "ARCTIC: A Dataset for Dexterous Bimanual Hand-Object Manipulation". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2023.

[21] Sergey Prokudin, Christoph Lassner, and Javier Romero. "Efficient Learning on Point Clouds with Basis Point Sets". In: *IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019.

[22] Javier Romero, Dimitrios Tzionas, and Michael J. Black. "Embodied Hands: Modeling and Capturing Hands and Bodies Together". In: *ACM Transactions on Graphics (Proc. SIGGRAPH Asia)* 36.6 (2017), 245:1–245:17. DOI: 10.1145/3130800.3130883. URL: https://doi.org/10.1145/3130800.3130883.

[23] Younghyo Park and Pulkit Agrawal. *Using Apple Vision Pro to Train and Control Robots*. 2024. URL: https://github.com/Improbable-AI/VisionProTeleop.

[24] John Schulman et al. "Proximal Policy Optimization Algorithms". In: *arXiv preprint arXiv:1707.06347* (2017). arXiv: 1707.06347 [cs.LG]. URL: https://arxiv.org/abs/1707.06347.

[25] Diederik P. Kingma and Jimmy Ba. "Adam: A Method for Stochastic Optimization". In: *International Conference on Learning Representations (ICLR)*. 2015. arXiv: 1412.6980 [cs.LG]. URL: https://arxiv.org/abs/1412.6980.

[26] Viktor Makoviychuk et al. "Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning". In: *arXiv preprint arXiv:2108.10470* (2021). arXiv: 2108.10470 [cs.RO]. URL: https://arxiv.org/abs/2108.10470.

A Appendix

A.1 Bottle Cap Data Distribution

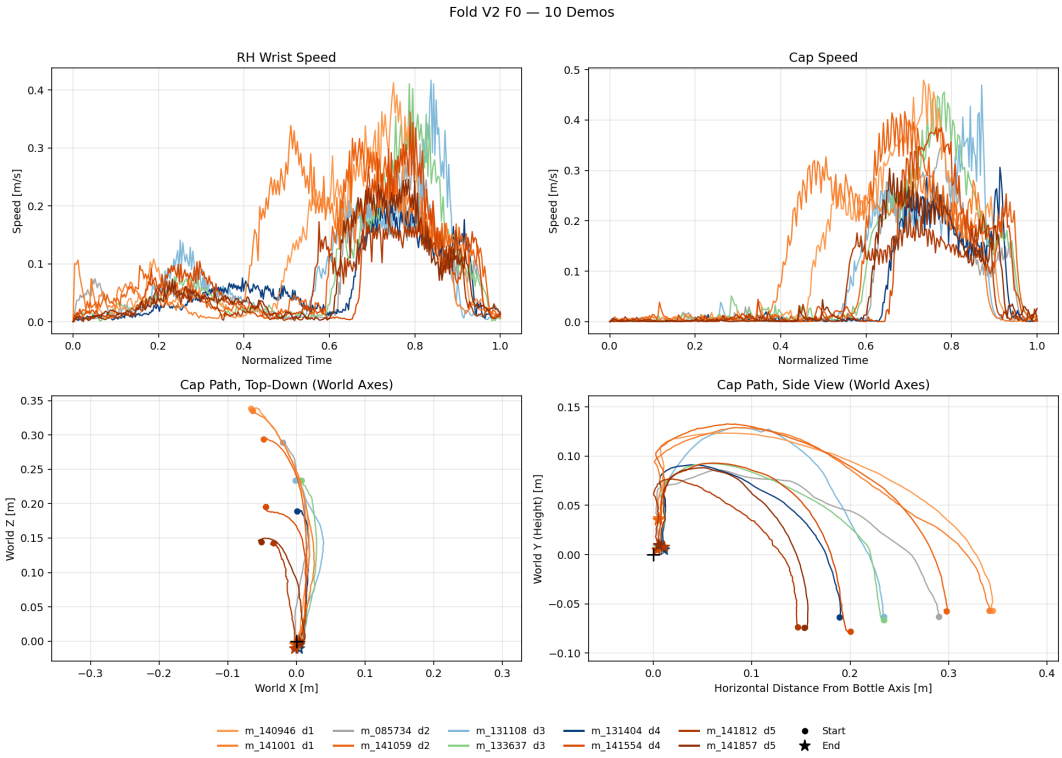

Figure 8: Reach demonstrations of cross-validation fold F0 (10 demonstrations, two per reach distance). Top: right-wrist and cap speed against normalized time, computed with the causal estimator used for the training targets. Bottom left: cap path viewed from above, expressed in world axes about the burner. Bottom right: the same paths as horizontal distance from the burner axis against height. Color denotes capture session; markers indicate the start (circle) and end (star) of each trajectory.

Velocity estimation. Reference velocities are computed with the same estimator used to generate the tracking targets at training time, so that the quantities plotted are exactly those the policy is required to reproduce. Linear velocities are computed causally in two steps. The position signal is first low-pass filtered with a forward-only exponential moving average, seeded at the initial sample,

$$\tilde{p}[t] = \alpha\, p[t] + (1-\alpha)\, \tilde{p}[t-1], \qquad \tilde{p}[0] = p[0], \qquad \alpha = 0.3, \tag{8}$$

and the smoothed signal is then differenced backward in time,

$$v[t] = \frac{\tilde{p}[t] - \tilde{p}[t-1]}{\Delta t}, \qquad v[0] = 0, \qquad \Delta t = \tfrac{1}{60}\,\mathrm{s}, \tag{9}$$

where Δt is the nominal capture period rather than the measured timestamps. Smoothing before differentiating avoids amplifying noise through the derivative. The estimator uses only past and current frames, so it is realizable in real time and matches the live streaming interface exactly: an offline target is identical to what would be observed online at the same instant. A non-causal alternative (central differences followed by Gaussian smoothing) yields a cleaner but unrealizable signal and is not used here. One consequence for the interpretation of the speed panels is that the plotted curves carry the phase lag of the causal filter.

Spatial organization of the demonstrations. The lower panels of Figure 8 show the cap trajectory relative to the burner. The demonstrations span five reach distances from the burner, with slightly varying start positions in x at each distance.

A.1.1 Observation space

Observations are a dictionary of three groups, built per hand and concatenated for the bimanual policy. `proprioception` and `privileged` are pure live simulator state — where the robot and object *are*; `target` carries the demonstration intent — where they *should be* one step ahead — with the δ-prefixed entries being the difference between the two. The one exception is `object-to-joint distances`, a live contact-affordance feature grouped with `target` because it is read by the residual policy alone, not because it carries demonstration intent. Table 7 lists the layout for the Inspire hand ($n_d = 12$ actuated joints, $n_b = 18$ rigid bodies, 5 contact links — thumb distal plus four intermediate phalanges).

Group	Component	Symbolic	Dim	Source
`proprioception`	joint positions q	n_d	12	Sim
	$\cos q$, $\sin q$	$2n_d$	24	Sim
	wrist position (zeroed)	3	3	—
	wrist orient., lin./ang. velocity	10	10	Sim
	subtotal	$13 + 3n_d$	**49**	
`privileged`	joint velocities $\dot{q}$	n_d	12	Sim
	object position (wrist frame), quat.	7	7	Sim
	object lin./ang. velocity	6	6	Sim
	contact-link force + magnitude	5×4	20	Sim
	object CoM (wrist frame)	3	3	Sim + Static
	object weight	1	1	Static
	subtotal	$n_d + 37$	**49**	
`target`	wrist pose target†	23	23	AVP
	joint pos. target‡	$9(n_b - 1)$	153	AVP
	object pose target†	23	23	OT
	object-to-joint distances	n_b	18	Sim
	ref. fingertip–object distances	5	5	AVP + OT
	object shape encoding (BPS)	128	128	Static
	subtotal	$170 + 10n_b$	**350**	
Total, per-hand policy input			**448**	
Total, bimanual policy input			**896**	

Table 7: Per-hand observation space for the Inspire hand. $^\dagger\delta$pos, vel, δvel, quat, δquat, ω, $\delta\omega$. $^\ddagger\delta$pos, vel, δvel per non-base link. δ = demonstration minus simulation. **Source**: Sim = live PhysX state; AVP = Apple Vision Pro hands; OT = OptiTrack objects, Static = object-model constant.

A.2 Data Augmentation

Each demonstration is a per-hand trajectory of wrist pose, 21 MANO keypoints, object pose, and the corresponding linear and angular velocities over T frames. All spatial augmentations spin the scene around a vertical axis, so objects stay upright on the table. Each augmentation

Table 8: Demonstration augmentations. All rotations are about the world Z axis. LH and RH denote the left and right hand demonstration (hand plus the object it manipulates).

Augmentation	Bodies moved	Pivot c_t	Angle
RH about LH object	RH hand + RH object	LH object center, per frame	$\mathcal{U}(-15°, +15°)$, constrained
RH about RH object	RH hand (object spins in place)	RH object center, per frame	$\mathcal{U}(-15°, +30°)$
LH about LH object	LH hand + LH object (rigidly)	LH object center, per frame	$\mathcal{U}(-15°, +30°)$
Object yaw	object orientation only	object's own center	$\mathcal{U}(-90°, +90°)$, per hand
Keypoint noise	wrist + MANO joints	—	additive $\mathcal{U}(-\sigma, +\sigma)$

is the same rigid map applied about a (possibly time-varying) pivot c_t:

$$p_t \;\mapsto\; R\,(p_t - c_t) + c_t, \qquad q_t \;\mapsto\; R\,q_t, \qquad \dot{p}_t \;\mapsto\; R\,(\dot{p}_t - \dot{c}_t) + \dot{c}_t, \qquad \omega_t \;\mapsto\; R\,\omega_t, \tag{10}$$

where $R = R_z(\theta)$. The velocity carries the $\dot{c}_t$ correction because the pivot itself moves with the object. The variants differ only in *which bodies are transformed* and *what serves as the pivot*; they are summarized in Table 8.

Augmentation Types

- **RH about LH object** is the only augmentation that varies the *inter-hand* geometry, letting the right hand approach the object held by the left hand from a different direction. This is the coordination variable the residual policy must generalize over in the capping task.
- **RH about RH object** orbits the right hand around the object it already holds, leaving that object's position unchanged, and so varies the grasp approach angle only.
- **LH about LH object** rotates the hand and object together, rigidly, preserving the demonstrated grasp and varying only the pose at which the object is presented.
- **Object yaw** rotates the object's orientation while leaving the hand fixed, deliberately breaking the demonstrated grasp alignment so that the policy must cope with an object presented at an unseen yaw.
- **Keypoint noise** emulates hand-pose-estimator and motion-capture error, drawn uniformly from $\mathcal{U}(-\sigma, +\sigma)$ with $\sigma = 0.5\,\text{cm}$.

Constrained sampling for the RH-about-LH-object rotation. Rotating the right hand and cap about the bottle shifts where the right hand starts, and that shift is bounded to at most $\delta_{\max} = 5\,\text{cm}$, as illustrated in Figure 9. Because the same rotation angle displaces a distant starting point further than a nearby one, the angle that keeps the shift under 5 cm is different for every demonstration: a demonstration that starts far from the bottle is capped at a small angle, one that starts close can rotate much further before reaching the same 5 cm shift. The yaw is therefore drawn per demonstration rather than fixed for the whole scene. Only the start is bounded; the trajectory is free to diverge further later, which is what gives the augmentation its effect.

(a) Far start: long lever arm, small angle.

(b) Near start: short lever arm, large angle.

Figure 9: Top-down view of the RH-about-LH-object rotation. The same 5 cm bound on the starting-position shift corresponds to a different rotation angle θ depending on how far the demonstration starts from the bottle.

Random action masking Following TeleDexter [16], a randomly chosen subset of joints is periodically frozen at its previously commanded target, so that the policy cannot assume that every degree of freedom responds on schedule. This is the only augmentation acting on the *action* channel rather than on the observation. At each control step, an environment with no active mask starts one with probability p; a fresh mask selects n degrees of freedom per hand uniformly without replacement and holds them for $d \sim \mathcal{U}\{1, d_{\max}\}$ steps. The masking is applied *after* the action moving average and joint-limit clamp, so what is frozen is the final commanded joint target, and the held value is written back as the previous target so that a multi-step freeze holds one command throughout rather than drifting. The maximum freeze duration $d_{\max}$ is annealed over T_{ramp} control steps,

$$\sigma(t) = 1-(1-\sigma_{\min}) \min\Big(\tfrac{t}{T_{\text{ramp}}}, 1\Big), \qquad d_{\max}(t) = 1+(D_{\max}-1)\,\frac{1-\sigma(t)^3}{1-\sigma_{\min}^3}, \qquad \sigma_{\min} = 0.7, \tag{11}$$

so the policy is always exposed to brief stale commands early in training; as $d_{\max}$ grows, single-step freezes continue to occur at full frequency while the mean freeze duration increases. Masks are drawn independently at each control step, so at any instant the batch spans a spectrum of partially stale joint commands; they are cleared on reset so a fresh episode never inherits one.

Hyperparameters The policy is an encoder followed by a shared actor–critic trunk. The encoder concatenates the three observation groups (Table 7) and compresses the resulting 896-d vector to a 256-d latent through a three-hidden-layer MLP of width 512 with Swish activations. The two frozen imitators' base actions are appended to this latent rather than passed through the encoder, so the residual sees them at full resolution; the resulting 292-d feature is processed by a four-layer trunk of widths $[256, 512, 128, 64]$ with ELU activations, shared between the policy and value heads. Two linear heads emit the action mean and the state value, with a state-independent learned log-standard-deviation. Full hyperparameters are listed in Table 9.

Hyperparameter	Value
Network	
Encoder hidden sizes	$[512, 512, 512]$
Encoder output dim	256
Encoder activation	Swish
MLP hidden sizes	$[256, 512, 128, 64]$
Activation	ELU
PPO	
Learning rate	2×10^{-4}
LR schedule	warm-up, then adaptive
KL threshold	0.008
Num opt-epochs	5
Minibatch size	4,096
Horizon length	32
Discount (γ)	0.99
GAE λ	0.95
Clip range (ϵ)	0.2
Max grad norm	1.0
Bounds-loss coef.	10^{-4}
Parallel envs	8,192
Augmentation	
Action-mask probability p	0.15
Masked DoFs per hand n	3
Maximum freeze duration $D_{\max}$[‡]	10 / 4
Mask ramp T_{ramp}	64,000 control steps
Ramp floor $\sigma_{\min}$	0.7
Reward — weight w (decay rate λ)	
Wrist position / orientation	0.1 (40) / 0.6 (1)
Thumb / index fingertip	0.9 (100) / 0.8 (90)
Middle / ring / pinky fingertip	0.75 (80) / 0.6 (60) / 0.6 (60)
Level-1 / level-2 joints	0.5 (50) / 0.3 (40)
Object position / orientation[†]	10.0 (80) / 10.0 (10)
Object lin. / ang. velocity[†]	0.1 (1) / 0.1 (1)
Wrist lin. / ang. velocity	0.1 (1) / 0.05 (1)
Joint velocity	0.1 (1)
Joint / wrist power	0.5 (10) / 0.5 (2)
Fingertip contact force w_c	1.0 ($\lambda_c = 1$)
Contact ramp ξ_c	$3 \rightarrow 2\,\text{cm}$

Table 9: Hyperparameters of the residual policy; reward weights are per hand. [†]Scaled by the hand's object share: 1 normally, splitting to sum to 1 for a shared object. [‡]Bottle-capping and brush-flipping respectively.

A.3 Augmentation Ablation

The five-fold cross-validation of Figure 6 is trained without trajectory augmentation. To isolate what the augmentation contributes we repeat it with the object-yaw augmentation enabled and nothing else changed (Figure 10): the same stratified split, the same 128 episodes scored per held-out demonstration, and the same evaluation tolerances.

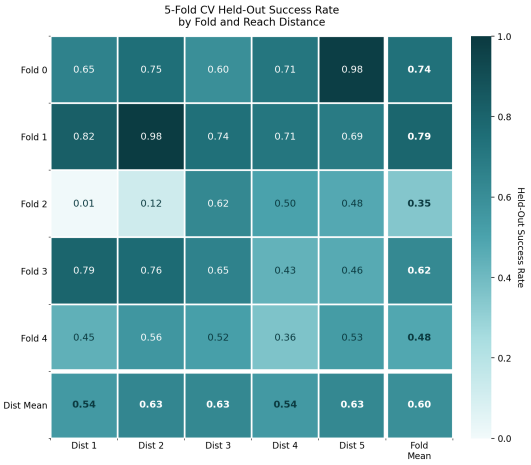

Figure 10: Five-fold cross-validation with the object-yaw augmentation enabled. Compare with Figure 6, which is otherwise identical but unaugmented.

The augmentation improves distance invariance more than mean success rate. The mean over all folds and distances rises from 0.56 to 0.60. The distance marginals change more: unaugmented, success broadly falls with reach distance (0.50, 0.42, 0.52, 0.64, 0.70, farthest to nearest; spread 28 points); with object yaw the marginals flatten to 0.54–0.63 (spread 9), with per-distance changes of +0.21 and +0.11 at the second and third reaches and −0.10 and −0.07 at the two nearest. This flattening addresses the distance dependence Section 2.2 identifies as an obstacle to handling unseen operator trajectories.

Rotating the object about its own axis breaks the demonstrated grasp-object alignment: the burner is never replaced at the same orientation twice, so the policy cannot rely on the cap's recorded yaw.

Two caveats qualify this result. Only 13 of 25 cells and three of five folds improve. Fold-level variance increases: the spread of fold means widens from 0.25 (0.41–0.66) to 0.44 (0.35–0.79), driven largely by fold 2, whose farthest-reach cell collapses from 0.51 to 0.01. With only ten demonstrations per fold, one or two unusually hard ones are enough to swing a fold's mean, so individual fold numbers should not be read as evidence about the augmentation on their own.

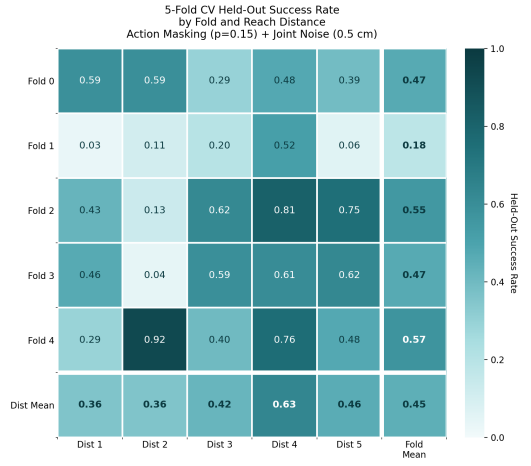

Figure 11: Five-fold cross-validation with random action masking ($p = 0.15$) and keypoint noise ($\sigma = 0.5\,\text{cm}$) enabled, object yaw disabled. Compare with Figure 6.

Action masking and keypoint noise, in isolation from object yaw, do not reproduce this benefit (Figure 11): the mean falls to 0.45, eleven points below the unaugmented baseline and fifteen below object yaw. The distance marginals stay as uneven as the baseline (spread 27 vs. 28 points), so the invariance gained under object yaw comes from that augmentation specifically, not from added training-time variability in general. Only 9 of 25 cells and two of five folds improve on the baseline, and fold 1 collapses to a mean of 0.18, three of its five distances below 0.20.

This is not evidence that action masking and keypoint noise are ineffective augmentations, only that this table is not the instrument that would show it. Action masking is designed for robustness to stale commanded joints, and keypoint noise for robustness to tracking-estimator error; neither targets reach-distance generalization, which is the one axis this cross-validation protocol scores. Object yaw is the only augmentation among the three that perturbs that specific degree of freedom, so it is the only one this table can credit. Any benefit the other two provide — to dropped-frame robustness or degraded-tracking robustness during live teleoperation — would not appear here even if it is real. A further limitation is that action masking and keypoint noise are combined in a single condition: the 0.45 mean cannot be attributed to one, the other, or their interaction, and with a single training seed per condition, fold 1's collapse cannot be distinguished from ordinary seed variance.

Read together, the three conditions support a narrower conclusion than "augmentation helps": of the techniques evaluated, only the one that directly perturbs the object's yaw improves the axis this five-fold protocol measures, and it does so by trading a small amount of individual-cell success for a substantial reduction in reach-distance dependence. Action masking and keypoint noise leave that dependence unchanged, which is consistent with neither being designed to address it. The protocol itself — Figure 6 onward — is therefore informative about generalization across held-out demonstrations at varying reach distance and approach angle specifically, and silent on any augmentation's value for failure modes, such as dropped commands or degraded tracking, that only manifest under live teleoperation.